INSTITUTE OF ENGINEERING & MANAGEMENT
Good Education, Good Jobs

UNIVERSITY OF ENGINEERING & MANAGEMENT
Good Education, Good Jobs

INSTITUTE OF ENGINEERING & MANAGEMENT, SALT LAKE
School of
UNIVERSITY OF ENGINEERING AND MANAGEMENT, KOLKATA

ZipGo – CLI-based Ride Booking System

ADVANCED PROGRAMMING (OOP) LABORATORY PROJECT
CODE: PCCCS495

SEMESTER-4 PROJECT REPORT

STREAM:	CSE(IoT)
COURSE:	B. TECH
SEMESTER:	4th
YEAR:	2nd
NAME - ROLL NUMBER:	**ANURAG DEB – 3**
COURSE INSTRUCTORS:	Dr. Susovan Jana, Dr. Swagatam Basu, Dr. Soumadip Biswas

ZipGo

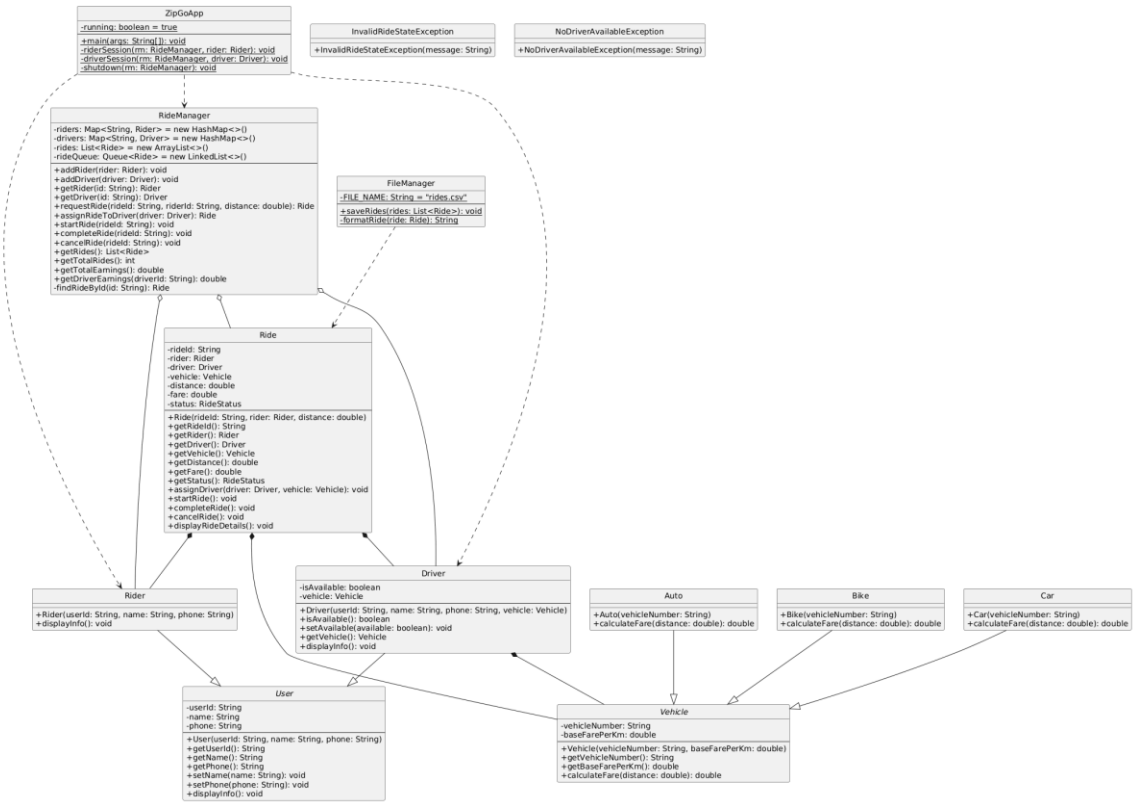
SPRING 2026

PROBLEM STATEMENT

Commuters in cities use ride-hailing apps to find a driver near them fast, but behind the scenes, the company must work hard to handle requests for rides, availability of drivers and the price of the ride. Many of these systems handle data management of structured data and coordinate activities among many people. The new ZipGo Ride Booking System simulates this system using a console-based Java application. When users request a ride through the system, they are assigned an available driver, a price that depends on vehicle type and distance, and the status of their ride. This project is designed using an object-oriented methodology, with a structured approach to each function involving abstraction, modularity and an extensible architecture within the system.

SYSTEM DESIGN

The system follows a layered, modular architecture separating core logic, data models, storage, and application interaction. It ensures clear separation of concerns and maintainability.



Major Classes Description:

- **RideManager:** Central controller managing ride requests, driver assignment, and lifecycle.
- **Ride:** Represents a ride entity with attributes such as rider, driver, distance, fare, and status.
- **Rider & Driver:** Represent system users with different roles and responsibilities.
- **Vehicle (Abstract Class):** Defines common vehicle behavior extended by Car and Bike.
- **FileManager:** Handles CSV-based persistence of ride data.

Package Structure:

- model → Entities (Ride, Rider, Driver, Vehicle)
- service → Business logic (RideManager)
- storage → File handling (FileManager)
- app → Main application (ZipGoApp)
- exception → Custom exceptions

Interaction Flow:

- Driver logs in and becomes available
- Rider requests a ride
- Request is added to queue
- Driver thread consumes request
- Ride progresses through states
- Data is saved after completion

OBJECT ORIENTED PROGRAMMING CONCEPTS DEMONSTRATED

Abstraction

Where used: Vehicle class
Why: To define a common structure for different vehicle types

Code Snippet:

```
public abstract class Vehicle {
    protected String vehicleNumber;
    public Vehicle(String vehicleNumber) {
        this.vehicleNumber = vehicleNumber;
    }
    public abstract double calculateFare(double distance);
}
```

Explanation:
Abstract class ensures all vehicle types implement fare calculation differently.

Inheritance

Where used: Car and Bike classes

Code Snippet:

```
public class Car extends Vehicle {
    public Car(String number) {
        super(number);
    }
    public double calculateFare(double distance) {
        return distance * 15;
    }
}
```

Explanation:
Car inherits common structure and overrides fare logic.

Polymorphism

Where used: Fare calculation

Code Snippet:

```
Vehicle v = new Car("WB01");
double fare = v.calculateFare(10);
```

Explanation:
Same method behaves differently depending on object type.

Encapsulation

Where used: Ride class fields

Code Snippet:

```
private double fare;

public double getFare() {

    return fare;

}
```

Explanation:
Data is protected and accessed via controlled methods.

Exception Handling

Where used: Ride state validation

Code Snippet:

```
if(status != RideStatus.ACCEPTED) {

    throw new InvalidRideStateException("Invalid state");

}
```

Explanation:
Ensures system integrity by preventing invalid transitions.

IMPLEMENTATION HIGHLIGHTS

Producer-Consumer Model

```
public synchronized Ride assignRideToDriver(Driver driver) {
    while (rideQueue.isEmpty()) {
        wait();
    }
    return rideQueue.poll();
}
```

Justification:
Ensures real-time assignment using thread-safe queue.

Wait-Notify Mechanism

```
notifyAll();
```

Justification:
Notifies driver threads when a new ride is requested.

Multithreading

```
new Thread(() -> {
    // driver logic
}).start();
```

Justification:
Simulates real-time independent driver behavior.

CSV Persistence

```
FileManager.saveRides(rm.getRides());
```

Justification:
Ensures data durability across executions.

TESTING & ERROR HANDLING

Test Cases

- Ride request with driver available
- Ride request with no driver
- Ride lifecycle completion
- Multiple ride requests

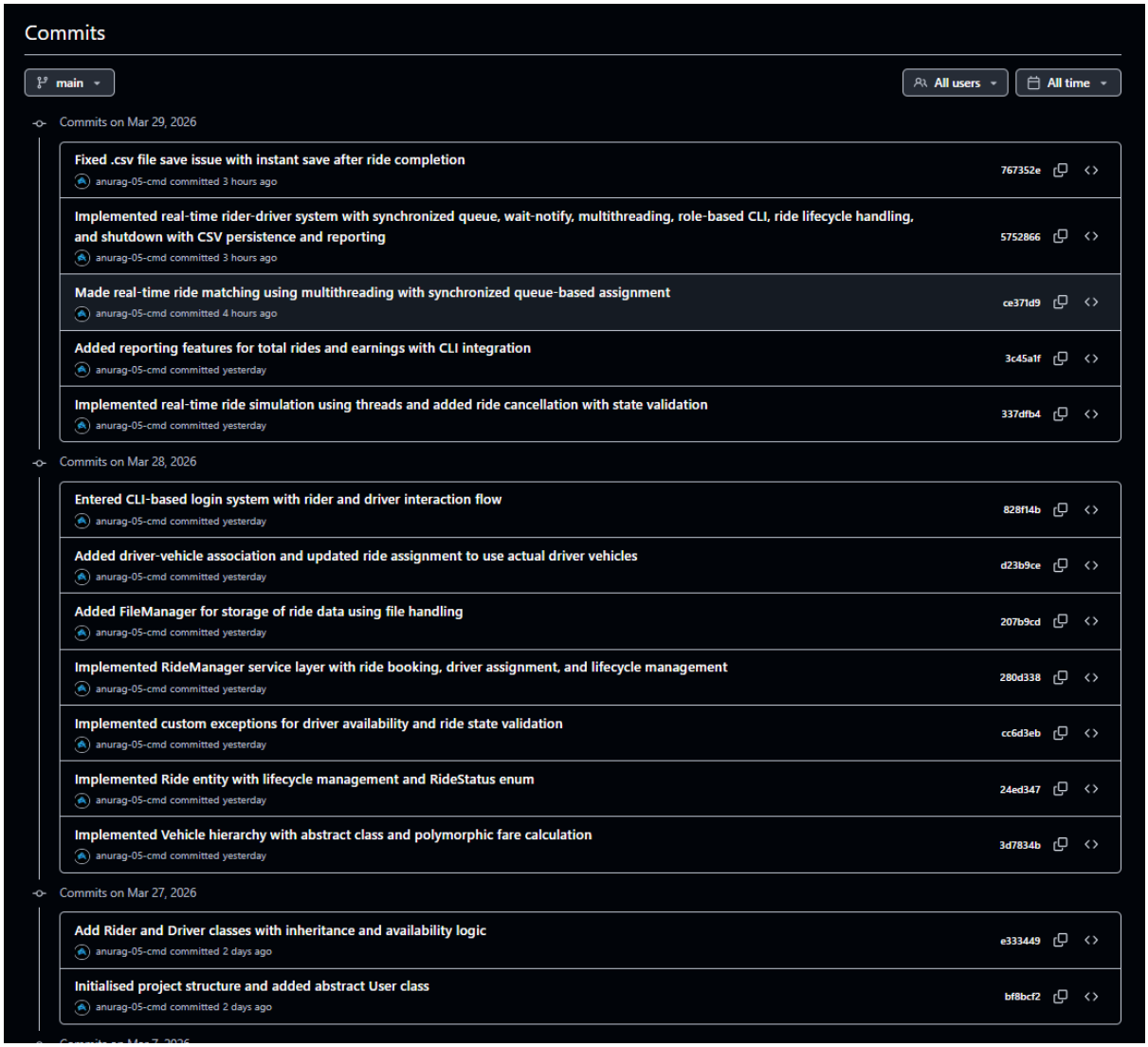
Edge Cases

- Driver offline
- Empty ride queue
- Invalid ride state transitions

Failure Scenarios

- No driver available → handled via message
- Invalid operations → exception thrown
- Interrupted threads → safely handled

GIT WORKFLOW



Explanation

Development was carried out incrementally:

- Initial setup of project structure
- Implementation of core classes
- Addition of OOP concepts
- Integration of multithreading
- Implementation of real-time behavior
- Addition of persistence and reporting

Each commit reflects logical progression and modular development.

CONCLUSION & FUTURE SCOPE

Conclusion

The ZipGo system successfully demonstrates object-oriented design, real-time processing using threads, and structured architecture. It effectively simulates a ride-hailing workflow while maintaining clean code practices and modularity.

Future Scope

- Multi-driver support with load balancing
- GUI-based interface
- Database integration
- Network-based multi-user system
- Advanced analytics and reporting

APPENDIX

Copy of Initial Project Proposal

ZipGo

A Console-Based Ride Booking Management System in Java

Anurag Deb – CSE(IoT) – 12024052019001 – Roll 3

Problem Statement:

Commuters in cities use ride-hailing apps to find a driver near them fast, but behind the scenes, the company must work hard to handle requests for rides, availability of drivers and the price of the ride. Many of these systems handle data management of structured data and coordinate activities among many people. The new ZipGo Ride Booking System simulates this system using a console-based Java application. When users request a ride through the system, they are assigned an available driver, a price that depends on vehicle type and distance, and the status of their ride. This project is designed using an object-oriented methodology, with a structured approach to each function involving abstraction, modularity and an extensible architecture within the system.

Target User:

- Riders who are requesting the transportation services
- Drivers who are providing the ride services
- System operators who are monitoring the rides and users

Core Features:

- Rider/Driver registration
- Vehicle management (Car, Bike, Auto)
- Ride Request and Driver Assignment
- Fare Calculation (by Vehicle Type and Distance)
- The lifecycle of a ride (Requested → Accepted → Completed/Canceled)
- Storing the Ride History via file handling
- Input validation and exception handling

OOP Concepts to be used:

Abstraction: All abstract classes will create the basic properties of the class for it to have common attributes & methods of specialized sub-classes. For example, User and Vehicle are both classes that can establish a base property for subclasses which derive from them.

Inheritance: The Driver and Rider are both classes that would derive from a shared User superclass and both Car, Bike, and Auto would all derive from Vehicle.

Polymorphism: All Vehicle subclass will override the fare calculation method & each will implement its pricing policy.

Exception Handling: We will create several custom exception classes to handle runtime errors. Examples include NoDriverAvailableException, InvalidRideStateException

Collections / Threads: We will use Java Collections (ArrayList, HashMap) to store & manage Users and Rides. We may use threads to simulate a ride's progress (e.g., at least one thread would be needed to allow the Car to move).

Architecture: This system uses a multi-tiered architecture to build objects on an object-oriented approach. The core objects are Riders, Drivers, Vehicles, and Rides, which represent the domain model for the system. All requests for rides come to a central RideManager; RideManager handles requests for rides, assigns drivers, and calculates fares. Data is stored and retrieved via a simple file management layer, so the application logic layer can be separate from the data management layer.

Rider Class/Driver Class/Vehicle Class → Ride Class → RideManager → File Storage Layer

Copy of the review mail receied (with APPROVED_MINOR)

PCCCS495 Proposal Review – APPROVED_MINOR

soumadip.biswas@ieminternational.ai
To: Anurag Deb

Reply | Reply all | Forward
Wed 2026-03-11 09:34

Dear Anurag Deb,

Project: ZipGo
Decision: APPROVED_MINOR
Score: 90

Strengths:
1. **Clear Problem and Scope:** The proposal clearly defines a relevant real-world problem and outlines a feasible scope for a university-level project, with well-chosen features that align with the problem.
2. **Strong OOP Foundation:** Explicitly lists and plans to apply core OOP concepts like Abstraction, Inheritance, Polymorphism, Exception Handling, and Collections, directly integrating them into the system's design.
3. **Sensible Domain Model and Architecture:** Identifies key domain objects (Riders, Drivers, Vehicles, Rides) and proposes a central 'RideManager' for business logic orchestration. The layered architecture with a file storage layer demonstrates good separation of concerns and modularity.
4. **Good Feature Set for OOP Practice:** Features such as 'Vehicle management' (ideal for inheritance/polymorphism), 'Fare Calculation' (potential for strategy pattern), and 'Ride lifecycle' (potential for state pattern) provide excellent opportunities to apply object-oriented design principles and patterns.
5. **Inclusion of Concurrency (Threads):** The intent to use threads is a valuable addition, offering a challenging but rewarding opportunity to explore concurrent programming in a meaningful way within the system (e.g., simulating driver availability or concurrent user interactions).

Improvements:
1. **Clarify 'Threads' Implementation:** While a strength, the proposal should provide more detail on *how* threads will be meaningfully integrated into a console-based application. For instance, whether it's for simulating background processes (like driver availability updates) or concurrent user interactions (if multiple simulated users are envisioned).
2. **Specify File Handling Approach:** Mentioning 'file handling' is good, but specifying the format (e.g., CSV, JSON, object serialization) and the strategy for robust persistent storage (e.g., handling data integrity, concurrent access if applicable) would enhance the proposal's completeness.
3. **Enhance System Operator Features:** Although 'monitoring' is mentioned, outlining specific reports, aggregated data views, or administrative functionalities for system operators would add more depth and value to this target user group.
4. **Consider Basic Authentication:** A typical ride-hailing system would include user authentication (login/logout) for riders and drivers. While not strictly an OOP concept, it's a common functional requirement that could be beneficial to consider for a more complete system.
5. **Refine 'Multi-tiered Architecture' Terminology:** While the separation of concerns is excellent, 'multi-tiered architecture' might be an overstatement for a console application with flat-file storage. 'Layered architecture' or 'modular design' might be more precise terms for the proposed structure.

Regards,
Course Instructor